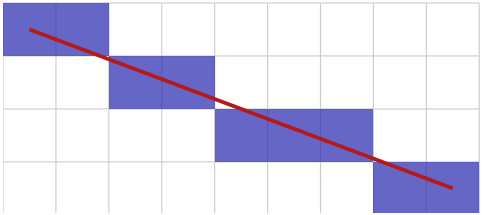
TP5 : tracer des segments, l'algorithme de Bresenham

R3.L01 — Représentations et codages des images

Fiche d'explication du TP, écrite à partir du sujet du TP5 (algorithmes donnés par le sujet) et du cours (CM diapos 42 et 47 : grille carrée, aliasing).

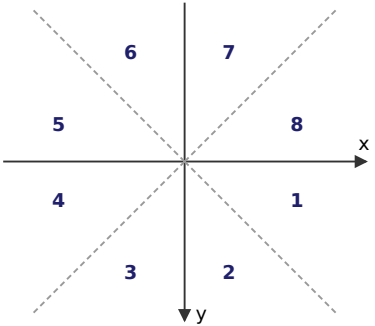
Code complet et testé : `PROJET_FINAL/Image.cpp` (ColorImage::line), `tp5.cpp`

Le problème : un segment sur une grille de pixels



Segment de (0, 0) à (8, 3) : la droite idéale (en rouge) passe entre les pixels. On doit choisir, **pour chaque colonne**, le pixel le plus proche de la droite.

Résultat inévitable : des « marches d'escalier ». C'est l'aliasing spatial du CM (diapo 47).



Les 8 **octants**. L'axe y est **vers le bas** dans une image : l'octant 1 ($0 \leq dy \leq dx$) descend donc doucement vers la droite.

Premier octant : l'algorithme du sujet

```
// 0 <= (y2-y1) <= (x2-x1)
const int c2 = 2 * (y2 - y1);
const int c1 = c2 - 2 * (x2 - x1);
int critere  = c2 - (x2 - x1);
int x = x1, y = y1;
while (x <= x2) {
  pixel(x, y) = couleur;
  if (critere >= 0) {   // changement de ligne horizontale
    y++;
    critere += c1;
  } else                // toujours la même ligne
    critere += c2;
  x++;                  // colonne suivante
}
```

- Pente entre 0 et 1 : **x avance de 1** à chaque colonne, y de 0 ou 1. Le **critère** dit si la droite idéale a dépassé le milieu entre les deux pixels possibles.
- Tout est **entier** : que des additions, ni division ni réel. D'où sa rapidité (1962 !).

Déroulé de (0, 0) à (8, 3)

dx = 8, dy = 3 ; c2 = 6 ; c1 = 6 − 16 = −10 ; départ : 6 − 8 = −2. Si critère < 0 on ajoute c2, sinon c1.

x	y	critère	décision	critère suivant
0	0	−2	même ligne	4
1	0	4	≥ 0 : y++	−6
2	1	−6	même ligne	0
3	1	0	y++	−10
4	2	−10	même ligne	−4
5	2	−4	même ligne	2
6	2	2	y++	−8
7	3	−8	même ligne	−2
8	3	−2	(fin)	

Ce sont les pixels de la page précédente. Les « 2 × » : on compare à un **demi**-pixel ; tout multiplier par 2 dx supprime le 0,5 et la fraction dy/dx.

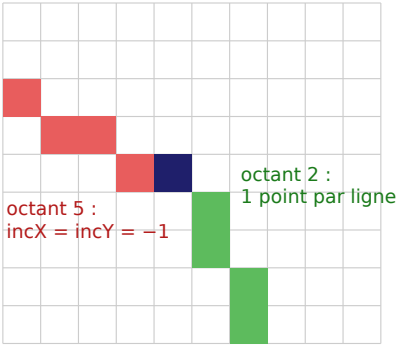
Deuxième octant, puis les 8 octants

Octant 2 : échanger les rôles de x et y

Si la pente est plus forte que 1 (longY ≥ longX), avancer d'une colonne à la fois laisserait des trous. On avance donc **d'une ligne à la fois** : `y++` à chaque tour, et `x++` seulement quand le critère est ≥ 0. Dans les formules, longX et longY échangent leur place.

Octants 3 à 8 (sujet)

- Remplacer `x++` par `x += incX`, avec `incX` = +1 ou −1 (idem pour y).
- longX et longY doivent rester **positives** : on prend leur valeur absolue.
- On ne peut plus tester `x <= x2` (x peut décroître) : on **compte les points**, de 0 à longX (ou longY).



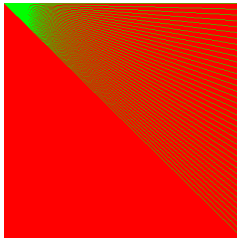
ColorImage::line : la version généralisée

```
void ColorImage::line(uint16_t x1, uint16_t y1,
                      uint16_t x2, uint16_t y2,
                      const Color pixel_value)
{
  if (x1 >= width || x2 >= width || y1 >= height || y2
>= height)
    throw std::out_of_range("line : extrémité hors de
l'image");
  int x = x1, y = y1;  // int : uint16_t ne sait pas
valoir -1
  int longX = x2 - x1;
  int longY = y2 - y1;
  const int incX = (longX >= 0) ? 1 : -1;
  const int incY = (longY >= 0) ? 1 : -1;
  if (longX < 0) longX = -longX;
  if (longY < 0) longY = -longY;
```

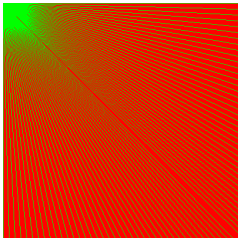
Le prototype est celui du sujet (`ushort = uint16_t`) ; on travaille ensuite en `int`, où x peut décroître.

```
  if (longY < longX) {          // un point par colonne
    const int c1 = 2 * (longY - longX);
    const int c2 = 2 * longY;
    int critere  = c2 - longX;
    for (int i = 0; i <= longX; i++) {
      pixel(uint16_t(x), uint16_t(y)) = pixel_value;
      if (critere >= 0) { y += incY; critere += c1; }
      else                critere += c2;
      x += incX;
    }
  } else {                      // un point par ligne
    const int c1 = 2 * (longX - longY);
    const int c2 = 2 * longX;
    int critere  = c2 - longY;
    for (int i = 0; i <= longY; i++) {
      pixel(uint16_t(x), uint16_t(y)) = pixel_value;
      if (critere >= 0) { x += incX; critere += c1; }
      else                critere += c2;
      y += incY;
    }
  }
}
```

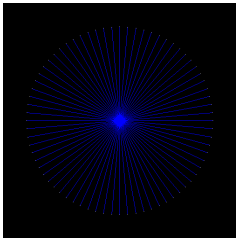
Les trois tests du sujet



Octant 1 : 400 × 400, fond rouge, de (0, 0) vers (399, y), y de 10 en 10.
identique à l'image du sujet



Octants 1 et 2 : on ajoute (0, 0) → (x, 399).
identique à l'image du sujet



8 octants : 500 × 500, du centre (250, 250) vers le cercle de rayon 200, tous les 5°.
Écart d'au plus 1 pixel sur quelques extrémités (arrondi de cos/sin).

```
for (int degres = 0; degres < 360; degres += 5) {
  double angle = degres * M_PI / 180.0;          // cos et sin attendent des RADIANS (<cmath>)
  uint16_t x = uint16_t(250 + 200 * std::cos(angle) + 0.5); // + 0.5 : arrondi
  uint16_t y = uint16_t(250 + 200 * std::sin(angle) + 0.5);
  im3.line(250, 250, x, y, bleu);
  im3.pixel(x, y) = blanc;                       // le point visé, en blanc comme sur la figure
}
```

Ce qui a été corrigé, et questions possibles

Version Gemini	Correction
`line(int16_t, int16_t, ...)` : **ce n'est pas le prototype du sujet** (`ushort`), alors que les noms et prototypes sont vérifiés.	`line(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2, const Color)` , calculs internes en `int` .
Points hors de l'image ignorés en silence.	Exception `out_of_range` , comme partout ailleurs.
Commentaire PPM « TP5 Bresenham - Eric Remy ».	Le nom de l'étudiant (c'est lui qui écrit l'image).
TGA, JPEG et rééchantillonnage absents du dossier TP5.	Un seul projet complet.
Points du cercle non dessinés en blanc.	Ajoutés, comme sur la figure.

1. **Pourquoi Bresenham est rapide ?** Uniquement des additions et des comparaisons d'entiers.
2. **Combien de pixels pour (0, 0) → (8, 3) ?** 9 : un par colonne, longX + 1.
3. **Et pour (0, 0) → (2, 7) ?** Octant 2 : un par ligne, 8 pixels.
4. **Pourquoi les variables internes en `int` ?** x peut décroître (incX = −1) et longX peut être négatif avant la valeur absolue : un `uint16_t` passerait à 65535.
5. **Piège classique** : `std::cos(5)` prend 5 **radians**, pas 5 degrés.